Task 2: Social Media Platform

Express + SQLite + JWT + vanilla JS. Profiles and bios, posts, comments, like toggle, follow toggle, following feed and explore.

GitHub repository name: **CodeAlpha_SocialMediaPlatform**

How to run

```
cd CodeAlpha_SocialMediaPlatform
npm install
npm start        # open http://localhost:3000  (Node 18+)
```

Overview

Features: profiles with bio, posts, comments, like toggle, follow toggle, following feed and explore feed. Tables: users, posts, comments, likes, follows.

Emoji in the UI are omitted from this listing only (the PDF font cannot show them).

CodeAlpha_SocialMediaPlatform/package.json

```
{ "name": "codealpha-social", "version": "1.0.0", "main": "server.js", "scripts": { "start": "node server.js" },
  "dependencies": { "express": "^4.19.2", "better-sqlite3": "*", "bcryptjs": "^2.4.3", "jsonwebtoken": "^9.0.2" } }
```

CodeAlpha_SocialMediaPlatform/auth.js

```
const jwt = require('jsonwebtoken'), bcrypt = require('bcryptjs');
const SECRET = process.env.JWT_SECRET || 'dev-secret-change-me';
const sign = u => jwt.sign({ id: u.id, username: u.username }, SECRET, { expiresIn: '7d' });
const verify = t => jwt.verify(t, SECRET);
const auth = (req, res, next) => {
  try { req.user = verify((req.headers.authorization || '').replace('Bearer ', '')); next(); }
  catch { res.status(401).json({ error: 'Login required' }); }
};
// Registers POST /api/register and /api/login (needs a `users(id, username, password)` table)
function authRoutes(app, db) {
  app.post('/api/register', (req, res) => {
    const { username, password } = req.body || {};
    if (!/^\w{3,20}$/.test(username || '') || (password || '').length < 6)
      return res.status(400).json({ error: 'Username 3-20 chars (letters/digits/_), password 6+ chars' });
    try {
      const id = db.prepare('INSERT INTO users(username,password) VALUES(?,?)').run(username,
          bcrypt.hashSync(password, 10)).lastInsertRowid;
      res.json({ token: sign({ id, username }), username });
    } catch { res.status(409).json({ error: 'Username already taken' }); }
  });
  app.post('/api/login', (req, res) => {
    const { username, password } = req.body || {};
    const u = db.prepare('SELECT * FROM users WHERE username=?').get(username || '');
    if (!u || !bcrypt.compareSync(password || '', u.password)) return res.status(401).json({ error: 'Wrong username
        or password' });
    res.json({ token: sign(u), username: u.username });
  });
}
module.exports = { sign, verify, auth, authRoutes };
```

CodeAlpha_SocialMediaPlatform/server.js

```
const express = require('express'), Database = require('better-sqlite3');
const { auth, authRoutes } = require('./auth');
const db = new Database('social.db');
db.exec(`
CREATE TABLE IF NOT EXISTS users(id INTEGER PRIMARY KEY, username TEXT UNIQUE, password TEXT, bio TEXT DEFAULT '');
CREATE TABLE IF NOT EXISTS posts(id INTEGER PRIMARY KEY, user_id INTEGER, body TEXT, created TEXT DEFAULT
    CURRENT_TIMESTAMP);
CREATE TABLE IF NOT EXISTS comments(id INTEGER PRIMARY KEY, post_id INTEGER, user_id INTEGER, body TEXT);
CREATE TABLE IF NOT EXISTS likes(post_id INTEGER, user_id INTEGER, PRIMARY KEY(post_id,user_id));
CREATE TABLE IF NOT EXISTS follows(follower_id INTEGER, followee_id INTEGER, PRIMARY KEY(follower_id,followee_id));`);
const app = express(); app.use(express.json()); app.use(express.static('public')); authRoutes(app, db);

const Q = 'SELECT p.id,p.body,p.created,u.username FROM posts p JOIN users u ON u.id=p.user_id';
const shape = (rows, uid) => rows.map(p => ({
  ...p,
  likes: db.prepare('SELECT COUNT(*) c FROM likes WHERE post_id=?').get(p.id).c,
  liked: !!db.prepare('SELECT 1 FROM likes WHERE post_id=? AND user_id=?').get(p.id, uid),
  comments: db.prepare('SELECT c.body,u.username FROM comments c JOIN users u ON u.id=c.user_id WHERE post_id=? ORDER
      BY c.id').all(p.id)
}));
app.get('/api/feed', auth, (req, res) => {
```

```
  const all = req.query.all;
  const sql = Q + (all ? '' : ' WHERE p.user_id=? OR p.user_id IN (SELECT followee_id FROM follows WHERE
      follower_id=?)') + ' ORDER BY p.id DESC LIMIT 50';
  res.json(shape(db.prepare(sql).all(...(all ? [] : [req.user.id, req.user.id])), req.user.id));
});
app.post('/api/posts', auth, (req, res) => {
  const body = (req.body.body || '').trim();
  if (!body || body.length > 500) return res.status(400).json({ error: 'Post must be 1-500 chars' });
  res.json({ id: db.prepare('INSERT INTO posts(user_id,body) VALUES(?,?)').run(req.user.id, body).lastInsertRowid });
});
app.post('/api/posts/:id/like', auth, (req, res) => {
  const a = [req.params.id, req.user.id];
  if (!db.prepare('SELECT 1 FROM posts WHERE id=?').get(a[0])) return res.status(404).json({ error: 'No such post' });
  db.prepare('SELECT 1 FROM likes WHERE post_id=? AND user_id=?').get(...a)
    ? db.prepare('DELETE FROM likes WHERE post_id=? AND user_id=?').run(...a)
    : db.prepare('INSERT INTO likes VALUES(?,?)').run(...a);
  res.json({ ok: true });
});
app.post('/api/posts/:id/comments', auth, (req, res) => {
  const body = (req.body.body || '').trim();
  if (!body || body.length > 300) return res.status(400).json({ error: 'Comment must be 1-300 chars' });
  if (!db.prepare('SELECT 1 FROM posts WHERE id=?').get(req.params.id)) return res.status(404).json({ error: 'No such
      post' });
  db.prepare('INSERT INTO comments(post_id,user_id,body) VALUES(?,?,?)').run(req.params.id, req.user.id, body);
  res.json({ ok: true });
});
app.get('/api/users/:username', auth, (req, res) => {
  const u = db.prepare('SELECT id,username,bio FROM users WHERE username=?').get(req.params.username);
  if (!u) return res.status(404).json({ error: 'User not found' });
  const n = (sql) => db.prepare(sql).get(u.id).c;
  res.json({
    ...u, id: undefined,
    followers: n('SELECT COUNT(*) c FROM follows WHERE followee_id=?'), following: n('SELECT COUNT(*) c FROM follows
        WHERE follower_id=?'),
    isFollowing: !!db.prepare('SELECT 1 FROM follows WHERE follower_id=? AND followee_id=?').get(req.user.id, u.id),
    posts: shape(db.prepare(Q + ' WHERE p.user_id=? ORDER BY p.id DESC').all(u.id), req.user.id)
  });
});
app.post('/api/users/:username/follow', auth, (req, res) => {
  const u = db.prepare('SELECT id FROM users WHERE username=?').get(req.params.username);
  if (!u) return res.status(404).json({ error: 'User not found' });
  if (u.id === req.user.id) return res.status(400).json({ error: "You can't follow yourself" });
  const a = [req.user.id, u.id];
  db.prepare('SELECT 1 FROM follows WHERE follower_id=? AND followee_id=?').get(...a)
    ? db.prepare('DELETE FROM follows WHERE follower_id=? AND followee_id=?').run(...a)
    : db.prepare('INSERT INTO follows VALUES(?,?)').run(...a);
  res.json({ ok: true });
});
app.put('/api/me', auth, (req, res) => {
  db.prepare('UPDATE users SET bio=? WHERE id=?').run(String(req.body.bio || '').slice(0, 160), req.user.id);
      res.json({ ok: true });
});
app.listen(process.env.PORT || 3000, () => console.log('Social on http://localhost:3000'));
```

CodeAlpha_SocialMediaPlatform/public/index.html

```
<!doctype html><html><head><meta charset="utf-8"><meta name="viewport"
    content="width=device-width,initial-scale=1"><title>SocialAlpha</title><link rel="stylesheet"
    href="style.css"></head>
<body><header><b> SocialAlpha</b><span id="nav"></span></header><main id="app" style="max-width:640px"></main>
<script src="common.js"></script>
<script>
let cur = () => feed();
const run = f => { cur = f; return f(); };
function nav() {
  $('#nav').innerHTML = token ? `<a onclick="run(()=>feed())">Following</a><a
      onclick="run(()=>feed(true))">Explore</a><a onclick="run(()=>profile('${me}'))">Me</a><a
      onclick="logout()">Logout</a>` : '';
}
const postsHtml = ps => ps.map(p => `<div class="card"><b><a style="cursor:pointer;color:#2563eb"
    onclick="run(()=>profile('${esc(p.username)}'))">@${esc(p.username)}</a></b> <small>${esc(p.created)}</small>
  <p>${esc(p.body)}</p><button class="alt" onclick="like(${p.id})">${p.liked ? '' : ''} ${p.likes}</button>
  ${p.comments.map(c => `<div class="cm"><b>${esc(c.username)}</b>: ${esc(c.body)}</div>`).join('')}
  <input id="c${p.id}" placeholder="Write a comment, press Enter"
      onkeydown="if(event.key==='Enter')comment(${p.id})"></div>`).join('') || '<div class="card">Nothing here yet -
      post something or follow people from Explore.</div>';
async function feed(all) {
  if (!token) { $('#app').innerHTML = authBox(); return; }
  const posts = await api('/feed' + (all ? '?all=1' : ''));
  $('#app').innerHTML = `<div class="card"><textarea id="t" maxlength="500" placeholder="What's
      happening?"></textarea><button onclick="post()">Post</button></div>` + postsHtml(posts);
  cur = () => feed(all);
}
async function profile(name) {
  const u = await api('/users/' + name);
  $('#app').innerHTML = `<div class="card"><h2>@${esc(u.username)}</h2><p>${esc(u.bio) || '<i>No
      bio</i>'}</p><p><b>${u.followers}</b> followers . <b>${u.following}</b> following</p>` +
    (u.username === me ? `<input id="bio" maxlength="160" placeholder="Update your bio"><button
```

```
        onclick="saveBio()">Save bio</button>`
      : `<button onclick="follow('${u.username}')">${u.isFollowing ? 'Unfollow' : 'Follow'}</button>`) + '</div>' +
        postsHtml(u.posts);
}
const guard = f => async (...a) => { try { await f(...a); await cur(); } catch (e) { alert(e.message); } };
const post = guard(async () => { await api('/posts', 'POST', { body: $('#t').value }); });
const like = guard(id => api(`/posts/${id}/like`, 'POST'));
const comment = guard(id => api(`/posts/${id}/comments`, 'POST', { body: $('#c' + id).value }));
const follow = guard(n => api(`/users/${n}/follow`, 'POST'));
const saveBio = guard(() => api('/me', 'PUT', { bio: $('#bio').value }));
nav(); feed();
</script></body></html>
```

CodeAlpha_SocialMediaPlatform/public/common.js

```
const $ = s => document.querySelector(s);
let token = localStorage.token || '', me = localStorage.me || '';
async function api(path, method = 'GET', body) {
  const r = await fetch('/api' + path, { method, headers: { 'Content-Type': 'application/json', Authorization:
      'Bearer ' + token }, body: body ? JSON.stringify(body) : undefined });
  const d = await r.json().catch(() => ({}));
  if (r.status === 401 && token) { localStorage.clear(); location.reload(); }
  if (!r.ok) throw new Error(d.error || 'Something went wrong');
  return d;
}
async function authSubmit(kind) {
  try { const d = await api('/' + kind, 'POST', { username: $('#u').value, password: $('#p').value });
    token = localStorage.token = d.token; me = localStorage.me = d.username; location.reload();
  } catch (e) { alert(e.message); }
}
function logout() { localStorage.removeItem('token'); localStorage.removeItem('me'); location.reload(); }
const esc = s => String(s ?? '').replace(/[&<>"']/g, c => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;',
    "'": '&#39;' }[c]));
const authBox = () => `<div class="card narrow"><h3>Login / Register</h3><input id="u" placeholder="username"><input
    id="p" type="password" placeholder="password (6+ chars)"><button onclick="authSubmit('login')">Login</button>
    <button class="alt" onclick="authSubmit('register')">Register</button></div>`;
```

CodeAlpha_SocialMediaPlatform/public/style.css

```
*{box-sizing:border-box}body{margin:0;font-family:system-ui,sans-serif;background:#f4f5f7;color:#222}
header{display:flex;justify-content:space-between;align-items:center;padding:12px
    18px;background:#1f2937;color:#fff;position:sticky;top:0;z-index:5}
header a{color:#cbd5e1;margin-left:14px;text-decoration:none;cursor:pointer}header a:hover{color:#fff}
main{max-width:1000px;margin:18px auto;padding:0 12px}
.card{background:#fff;border-radius:10px;padding:14px;margin-bottom:12px;box-shadow:0 1px 3px
    #0002}.narrow{max-width:360px;margin:40px auto}
input,textarea,select{width:100%;padding:9px;margin:5px 0;border:1px solid #cbd5e1;border-radius:6px;font:inherit}
button,.btn{background:#2563eb;color:#fff;border:0;padding:8px
    14px;border-radius:6px;cursor:pointer;font:inherit;display:inline-block}
button.alt,.btn.alt{background:#64748b}button:hover{opacity:.9}
.grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(210px,1fr));gap:12px}.big{font-size:56px;text-align:c
    enter}
.cols{display:grid;grid-template-columns:repeat(3,1fr);gap:12px}@media(max-width:700px){.cols{grid-template-columns:1f
    r}}
.cm{background:#f1f5f9;border-radius:6px;padding:6px 9px;margin:5px 0;font-size:14px}
#toast div{position:fixed;right:14px;bottom:14px;background:#111;color:#fff;padding:10px 14px;border-radius:8px}
```